# Mikhail Razakov

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

## C++ / Compiler Systems & Program Analysis

Compiler-tooling and program-analysis engineer working mainly in C++/LLVM. At MCST, I worked with LLVM 22 and C++23; independent projects cover interprocedural IR transformations, memory-dependence analysis, and an educational pipeline from SSA-like IR through register allocation to SysV x86-64.

## EXPERIENCE

**2026 — present**

### ISP RAS — Static Analysis Engineer

- Contribute to SharpChecker, a C#/.NET static-analysis platform developed at ISP RAS.

**July — August 2026**

### MCST — Compiler Engineering Intern

- Implemented an LLVM 22 LICM pass with loop analysis, side-effect and speculative-safety checks, and hoisting of loop-invariant instructions to a preheader.
- Developed a conservative interprocedural global-IV prototype with APInt affine evolution, transitive call effects, a separate legality phase, and an LLVM IR transform; unsupported CFG and call cases are left unchanged.

## PROJECTS

### LLVM Interprocedural Global IV Optimization

An LLVM 22 pass that localizes supported integer globals inside loops. It models affine evolution with APInt, tracks transitive call effects, and changes IR only after a separate legality check.

29/29 CI regression cases cover successful transforms and unsupported-IR rejection, LLVM Verifier checks, a second pass run, and before/after execution for runnable cases.

### PS-form Memory Dependence Analyzer

Conservative analysis of parametric memory-access overlap using normalization, interval and modular/GCD constraints, exact affine cases, and guarded fallbacks.

Unproved cases return maybe. Small domains are checked by an independent exact oracle, with randomized/metamorphic generation and reproducible counterexamples.

### x86-64 Codegen & Register Allocation Lab

An educational compiler backend with SSA/CFG validation, liveness/interference, register allocation, phi lowering, and an x86-64 emitter.

Assignments are checked by a separate verifier; native code runs in a child process and is differentially compared with the IR interpreter.

## SKILLS

**C++ / toolchain**
C++23, C17, CMake, Linux, clang-tidy, ASan/UBSan, GitHub Actions

**Compiler analysis**
LLVM IR, CFG/SSA, dominance, loops, SCC, data flow, interprocedural effects, memory dependence

**Code generation**
SysV x86-64, liveness/interference, linear scan, phi lowering, IR interpretation

**Testing**
LLVM Verifier, differential/metamorphic tests, exact small-domain oracles, counterexample reproduction

## EDUCATION

HSE University — Software Engineering, 2026–2030

## RECOGNITION

**UniversalToolchain — Baltic Science and Engineering Competition**
First-degree diploma and the Grand Prize “Perfection as Hope” for UniversalToolchain.

**MEPhI Junior**
First-degree diploma, 96/100 for the UniversalToolchain project.

Moscow / remote